

Highlights

A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge

Mehdi Tarkian

- Research highlight 1
- Research highlight 2

A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge

Mehdi Tarkian

Division of Product Realization, Department of Management and Engineering, Linköping University, Linköping, 58183, Sweden

Abstract

Abstract text.

Keywords:

1. Introduction

Recent advances in large language models (LLMs) have intensified efforts to **capture** and **operationalize** institutional knowledge at scale. Engineering organizations, in particular, **must retrieve precise details about products, components, and processes** by cross-referencing specifications, standards, service manuals, engineering change orders (ECO/ECN), and part-revision histories—tasks that traditionally require years of **tacit know-how**. Retrieval-Augmented Generation (RAG) improves access to such knowledge, yet RAG pipelines **hallucinate, struggle with multi-document reasoning, and underperform on non-text modalities** common in engineering (dense tables, figures, CAD callouts, scanned PDFs) [1–3].

Advanced RAG. Methods such as RAPTOR build hierarchical representations of long documents and retrieve at multiple abstraction levels, improving cross-chunk coherence and reducing irrelevant context [4]. While RAPTOR helps with **concept-level queries** (e.g., “torque requirements across assemblies”), engineering settings still face persistent hurdles: (i) revision and change awareness (answers must reflect the correct revision/serial range and supersession rules); (ii) table/units fidelity (facts often live in tables and require robust parsing and normalization); (iii) cross-constraint reasoning across BOMs, ECOs, service bulletins, and standards; and (iv) traceability to exact clauses, figures, or table cells for audits [1,4].

Why tables and PDFs remain hard. Even with better retrieval, accuracy hinges on reliably extracting structure and numbers from PDFs. Large-scale studies show table detection and structure recognition remain challenging; dependable parsing and unit normalization often require specialized datasets and tools (e.g., PubTables-1M, GROBID/GROBID-quantities) that standard LLM pipelines rarely embed by default [5–7].

Agent frameworks mitigate some limitations by orchestrating tools, multi-step planning, and verification (e.g., ReAct-style reasoning-and-acting; tool-use learning) [8,9]. In engineering, however, no single agent suffices: depending on the domain and need, teams compose domain-specific agents—a table/units agent for numeric extraction and normalization, a revision/lineage agent for change control, a CAD/PLM agent for configuration, and a standards agent for normative rules—dramatically increasing the burden of tracking state, node/edge flows, retries, error routes, and observability. Frameworks such as LangGraph help, but productionizing typically still requires formal state design, graph wiring, heterogeneous tool integration (vector stores, OCR, table parsers, CAD/PLM APIs), persistence, evaluation, and replay [10–12].

This study presents a generic agentic framework that builds upon LangGraph by introducing a domain-agnostic control loop to standardize orchestration, validation, and state management across diverse application domains. The framework structures agentic reasoning into six coordinated stages—goal definition, strategy selection, agent execution, agent validation, strategy validation, and goal validation—ensuring that every reasoning step is both purposeful and verifiable.

A key innovation lies in its relational persistence model, where all orchestration artifacts—including goals, strategies, agent calls, parameters, outputs, and validation outcomes—are stored in an SQL-backed control plane rather than transient in-memory states. This approach unifies planning and execution while providing a coherent, queryable representation of the system’s behavior. The persistent state enables full observability, deterministic replay, and governance through audit-ready records. By externalizing orchestration policies and maintaining independence from specific retrieval or vector-store technologies, the framework achieves high modularity, traceability, and adaptability, significantly reducing setup complexity and operational overhead.

Empirical evaluations on engineering document retrieval and reasoning tasks demonstrate that this architecture maintains the factual accuracy ben-

efits of multi-step tool use and advanced retrieval methods—including hierarchical, RAPTOR-style retrieval—while significantly reducing hallucination rates through structured validation. Moreover, by externalizing orchestration policies into data rather than code, the framework minimizes integration overhead and ongoing maintenance effort compared to bespoke agent graphs with ad hoc state handling [1,3–4,8–12].

[1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401.

[2] Huang, L. et al. (2023). A Survey on Hallucination in Large Language Models. arXiv:2311.05232.

[3] (Context on limitations and recent coverage) LiveScience explainer on hallucinations and mitigation (2025).

[4] Parthasarathi, P. et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv:2401.18059; official repo.

[5] Smock, B., Pesala, R., Abraham, R. (2022). PubTables-1M: Towards Comprehensive Table Extraction from Unstructured Documents. CVPR. CVF Open Access

[6] Lopez, P. (GROBID). GROBID: Machine-learning PDF→TEI extraction (docs GitHub). grobid.readthedocs.io

[7] Galib, C. et al. (2019). Automatic Identification and Normalisation of Physical Measurements in Scientific Literature (GROBID-quantities). CIKM. ACM Digital Library

[8] Yao, S. et al. (2022–2023). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.

[9] Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

[10] LangGraph docs—workflows/agents overview.

[11] LangGraph persistence (checkpointers) documentation.

[12] LangGraph GitHub (feature overview) and site.

2. State of the Art

2.1. Agentic AI in Manufacturing

Agentic AI systems are characterized by autonomous decision-making, goal-oriented action, and coordination among multiple agents, often operating with limited supervision in dynamic environments [1, 2]. Unlike traditional AI rule-based or single-model classification—agentic AI uses rein-

forcement learning, multi-agent systems (MAS), and LLMs to continuously reason, plan, and act in response to changing conditions [3].

LLMs are used to extract design knowledge from engineering documents. Nicholson et al. (2025) evaluate GPT-4V (a vision-enabled LLM) on extracting information from 2D mechanical drawings. In one experiment, GPT-4V could describe a part and correctly read off five key dimensions from a drawing [4]. It then generated CAD scripts (in CadQuery, FeatureScript, OpenSCAD) to model the part, iteratively refining errors. This shows that VLMs can partially automate drawing interpretation and CAD generation, though the authors note that GPT-4V’s limited short-term memory required corrective feedback in multiple prompts [4]. In document knowledge management, Freire et al. (2024) build an LLM-based Q&A system that ingests a factory’s documentation and operator knowledge. Their system answers operator queries by searching manuals and past logs. A user study found that the LLM tool dramatically sped up information retrieval and problem resolution, though workers still preferred human mentors when available [5]. These results emphasize the promise of LLMs for mining engineering knowledge, as well as the ongoing need for careful validation of the answers they provide.

Another challenge is hallucination and factual grounding. VLMs like GPT-4V and GPT-4 are prone to “hallucinating” plausible-sounding but incorrect details [6]. Systems try to mitigate this by using external knowledge sources: for example, Gkournelos et al. embed the LLM in a digital twin and knowledge base. They explicitly give the agent access to “manufacturing proprietary knowledge bases” and live data via a digital twin, so it can verify its outputs [6]. A related issue is the limited context and memory of LLM agents. Most systems use one or two conversational agents with fixed context windows. Nicholson et al. found that even GPT-4V loses track of earlier details when prompted iteratively through many CAD corrections [4]. Singh et al. note that continuous processes (batch vs continuous) may require “more reprompts” and that LLMs’ finite context will limit how much of the system model can be processed at once [7]. They suggest RAG as a solution to feed the LLM only the necessary structured slices of the digital twin [7]. In practice, this means splitting knowledge into a database or memory store and having LLM agents fetch relevant facts as needed.

There is a challenge in integrating agentic AI with formal validation frameworks or redundant checks to catch erroneous agent outputs before they cause defects. The literature shows a rapid rise of multi-agent, LLM-

driven automation for engineering. These systems demonstrate the potential of agentic AI to improve flexibility and autonomy. Yet they also reveal gaps: **existing methods often scale only to narrow tasks, rely on fixed architectures,** and **incur hallucinations without grounding.** Future research must tackle these gaps by **designing truly dynamic agent systems, embedding structured knowledge (databases, digital twins) for LLMs, and building rigorous validation layers.** As multiple researchers observe, combining LLM reasoning with dedicated knowledge resources and modular agent coordination is essential for safe, scalable manufacturing automation

3. Methodology

This work introduces a generic agentic framework that **extends the capabilities of LangGraph** by abstracting orchestration into a domain-agnostic **control loop.** The **proposed architecture standardizes agent coordination, validation, and state management,** thereby substantially **reducing the cost and complexity of setup across different application domains.** **The framework operationalizes agentic reasoning through a structured six-stage process:** goal definition, strategy selection, agent execution, agent validation, strategy validation, and goal validation. The controller executes a six-stage loop (figure and bullets needed here):

Normalize user query into a Goal object with explicit success criteria, target variable(s), required evidence granularity (e.g., table cell citation), applicable revision scope, and acceptable uncertainty.

this should be itemised

Choose a strategy template (e.g., Revision-Aware Spec Lookup, Table-Grounded Numeric Extraction, Cross-Document Conflict Resolution). A strategy defines what kinds of agents may be invoked and in which order e.g. search, pars, normalize and verify.

Dynamically select one or more agents whose preconditions match the current state (e.g., product ID known vs. unknown) and **whose outputs advance the strategy’s targets** (e.g., “obtain torque value + units + source cell”).

Run local checkers for schema completeness, unit normalization, range plausibility, document/section anchors, and failure diagnostics. Failed checks trigger retries or alternate agents.

Test whether the collected artifacts meet the plan’s sufficiency conditions, e.g., at least two corroborating sources, consistent revisions, no unresolved

conflicts. If not, the controller loops back to step 3 within the same strategy or escalates to a broader strategy.

Apply the goal acceptance predicate to confirm correct revision and serial range, exact citations (cell/figure/paragraph), and uncertainty within thresholds. Only then is the answer finalized.

This loop decouples progress monitoring (validation gates) from capability invocation (agents), which makes the system robust to partial failures and amenable to targeted improvements.

A distinguishing characteristic of this framework is its relational persistence model. Instead of maintaining transient, in-memory states typical of conventional agent frameworks, all orchestration artifacts—goals, strategies, agent invocations, parameters, outputs, validations, and errors—are persisted in an SQL database. This design yields a coherent, queryable state representation that unifies planning and execution. The SQL-backed control plane simplifies observability through dashboards and lineage queries, supports systematic evaluation through deterministic replays, and facilitates governance through audit-ready traces. By decoupling orchestration from any specific vector store, retriever, or external tool provider, the system remains modular and adaptable to heterogeneous data environments.

3.1. Relational Control Plane

The Relational Control Plane (RCP) serves as the central orchestration database of the proposed framework, externalizing coordination logic from code into a normalized relational schema. It provides a generic and auditable foundation for agentic reasoning, particularly within engineering domains where precision, traceability, and validation are essential. Conceptually, the RCP enforces a clear separation between design-time policy and run-time execution. At design time, the library tables define the system’s capability space—strategies are represented as reusable plan templates, while agent functions are modeled as atomic operations with declared input–output contracts. At run time, corresponding instance tables record each concrete goal, strategy, and function invocation, including their parameters, outputs, validation outcomes, and diagnostic messages. This design allows behavior to be authored once and reused across multiple domains, while also transforming orchestration into persistent, versioned data that can be analyzed and refined without modifying the underlying control graph.

Upon receiving a query, the controller instantiates a goal record with explicit success criteria—such as revision scope, evidence granularity, or ci-

tation requirements—and selects an appropriate strategy instance according to stored policies in the RCP libraries (insert picture). The controller then dynamically dispatches one or more agents whose input–output contracts match the current system state. Typed contracts are enforced directly within the RCP: each function template defines its expected schema and post-conditions, while validation outcomes at the function, strategy, and goal levels are persisted as structured records in the instance tables. Progress through the loop is therefore validation-centric, with advancement permitted only when defined predicates—such as unit normalization, revision applicability, or citation exactness—are satisfied. All computational artifacts (including structured tables, retrieved passages, intermediate results, and evidence links) are stored within the RCP, whereas the orchestration graph itself maintains only lightweight references, such as the active goal, strategy, or function identifiers. This arrangement minimizes in-memory state, enables reproducible runs, and provides complete observability through standard relational queries.

Compared to conventional agentic frameworks—including basic Lang-Graph deployments in which logic and execution order are often hard-coded—the RCP introduces three fundamental advances. (bullets) Changes in execution order, eligibility, or capability preference (for example, inserting a lineage-checking step before finalization or prioritizing hierarchical retrieval over keyword search) are achieved through updates to policy data rather than through code rewrites. Every decision, artifact, and validation outcome is recorded, enabling deterministic replay, post hoc analysis, and quantitative evaluation of strategy performance. New capabilities or alternative implementations (e.g., retrievers, table parsers, or unit normalization tools) can be added by simply registering them within the RCP library, allowing existing strategies to incorporate them immediately without altering the orchestration graph.

Collectively, these properties deliver provenance and audibility by design, while significantly lowering the cost of adapting the framework across domains and document types.

The novelty of the proposed framework lies in its treatment of orchestration as data. Strategies, agent contracts, and validation rules are elevated to persistent, queryable entities within the RCP, while the graph controller functions primarily as an interpreter of these declarative policies. The RCP enables data-driven policy updates rather than architectural redesigns. Thus, providing a foundation of generality, audibility, and adaptability that current agentic frameworks lack.

3.2. Strategy Library

The Strategy Library defines a set of reusable, templated plans that encapsulate standardized patterns of tool usage, reasoning, and validation across different engineering contexts. Each strategy represents a distinct problem-solving blueprint that specifies four core components: targets, eligible agents, sufficiency conditions, and fallback policies.

The targets describe the evidence or variables required for successful completion of the goal, such as a numerical value, physical unit, revision identifier, or document citation. Eligible agents define the permissible computational modules and their preferred execution order, for example, a workflow that begins with search, proceeds to table parsing and unit normalization, and concludes with revision verification. The sufficiency conditions express the level of coverage and consistency necessary to consider a strategy complete, such as requiring two corroborating sources or one source explicitly documenting revision supersession. Finally, fallback policies specify adaptive behaviors when a strategy fails to meet its sufficiency threshold, such as expanding the search from a specific product to a broader component class or triggering hierarchical retrieval when local searches provide inadequate coverage.

3.3. Agent Library

The Agent Library contains modular, atomic capabilities that execute the functional operations defined within strategies. Each agent is designed as a self-contained, side-effect-free component with declared inputs, outputs, and post-conditions. Agents can be dynamically composed to fulfill the operational steps of a strategy without modification to the orchestration graph, ensuring both modularity and maintainability.

Typical agents include components for text retrieval, hierarchical retrieval, table extraction, unit normalization, revision and lineage checking, figure and callout linking, keyword expansion, table filtering, and lightweight evidence scoring. The text retrieval agent performs keyword- and semantic-based searches within document repositories, supporting constraints such as product identifiers, revision levels, or document classes. Hierarchical retrieval agents extend this functionality by constructing and traversing document trees to surface content at varying structural depths, thereby improving conceptual recall in complex documents. Table extraction and structuring agents identify tabular regions within PDFs, reconstruct headers and merged cells, and emit normalized data with explicit cell anchors. Unit normalization agents detect and convert physical quantities into canonical units while

maintaining source provenance. Revision and lineage checking agents trace supersession chains, serial-range applicability, and effective dates to ensure that data conforms to the correct revision state. Additional agents manage figure or callout linkage, keyword and entity expansion, and filtering of candidate tables or rows using rule-based or learned relevance functions. Lightweight evidence scoring agents assess the adequacy of the gathered evidence relative to the strategy’s targets, guiding the system toward sufficiency or escalation when necessary.

Together, these agents form the operational substrate of the framework. Each produces validated artifacts that feed directly into a strategy’s target set—comprising elements such as value, unit, revision, and citation. The strategy’s validation gates then determine whether these artifacts collectively satisfy the defined sufficiency conditions. If the conditions are unmet, the orchestration layer automatically invokes additional or alternative agents. Because all agents adhere to uniform interface specifications, individual components can be replaced or extended—such as substituting a standard retriever with a hierarchical variant—without any modification to the orchestration logic. This design principle ensures interoperability, robustness, and adaptability in multi-agent reasoning across diverse engineering document domains.

3.4. *In-Session Instantiation*

The Relational Control Plane (RCP) provides the persistent substrate that binds design-time definitions to run-time execution. At design time, the Strategy Library (Section 3.2) and the Agent Library (Section 3.3) declare the capability space in terms of reusable strategy templates and atomic agent functions, each with typed input–output contracts and post-conditions. At run time, these templates are instantiated as in-session records that capture the concrete evolution of a reasoning episode. For every user query, the controller creates a goal instance with explicit acceptance criteria and then materializes a strategy instance selected from the Strategy Library. Within this strategy context, the controller dynamically instantiates one or more agent functions from the Agent Library. Each instantiation records its parameterization, inputs consumed, outputs produced, and the results of function-, strategy-, and goal-level validations.

The in-session tables thus serve as a normalized execution ledger. Inputs and outputs are stored as first-class artifacts linked to their originating strategy and agent instances, together with citations, revision lineage, unit

normalizations, and diagnostic messages. The orchestration graph maintains only stable identifiers (e.g., current goal, strategy, and **function IDs**), while all heavy artifacts—retrieved passages, structured tables with cell anchors, intermediate calculations, and validation reports—reside in the RCP. This arrangement enables deterministic replay and post hoc inspection, supports fine-grained provenance from final answers back to specific clauses or table cells, and provides a coherent basis for evaluation and governance. Because the controller reads policies from the libraries and writes outcomes to the in-session ledger, the behavior can be adapted by updating library entries rather than rewriting orchestration logic. New strategies or agents become immediately available for instantiation once registered, and their inputs/outputs are automatically captured under the same schema. In effect, the RCP operationalizes a data-driven “goal $\rightarrow$ strategy $\rightarrow$ agent $\rightarrow$ validation” loop: strategies and **functions** defined in the libraries are realized as concrete, auditable instances, and all inputs, outputs, and validations are persistently recorded, ensuring consistent state management and traceable multi-agent reasoning across domains.

References

- [1] J. Lim, B. Vogel-Heuser, I. Kovalenko, Large language model-enabled multi-agent manufacturing systems, in: 2024 IEEE 20th International Conference on Automation Science and Engineering (CASE), IEEE, 2024, pp. 3940–3946.
- [2] K. Jafari Meimandi, G. Aránguiz-Dias, G. R. Kim, L. Saadeddin, M. J. Kochenderfer, The measurement imbalance in agentic ai evaluation undermines industry productivity claims, arXiv e-prints (2025) arXiv-2506.
- [3] P. A. Olujimi, P. A. Owolawi, R. C. Mogase, E. V. Wyk, Agentic ai frameworks in smmes: A systematic literature review of ecosystemic interconnected agents, AI 6 (6) (2025) 123.
- [4] C. Picard, K. M. Edwards, A. C. Doris, B. Man, G. Giannone, M. F. Alam, F. Ahmed, From concept to manufacturing: Evaluating vision-language models for engineering design, Artificial Intelligence Review 58 (9) (2025) 288.

- [5] S. K. Freire, C. Wang, M. Foosherian, S. Wellsandt, S. Ruiz-Arenas, E. Niforatos, [Knowledge sharing in manufacturing using large language models: User evaluation and model benchmarking](#), arXiv preprint arXiv:2401.05200 (2024).
- [6] C. Gkournelos, C. Konstantinou, S. Makris, [An llm-based approach for enabling seamless human-robot collaboration in assembly](#), CIRP Annals 73 (1) (2024) 9–12.
- [7] M. Singh Gill, J. Vyas, A. Markaj, F. Gehlhoff, M. Mercangöz, Leveraging llm agents and digital twins for fault handling in process plants, arXiv e-prints (2025) arXiv-2505.